

Chapter 1

1.1 Level 1

Question 1: `std::print`

What does `std::print` provide compared to `printf` or `std::cout`?

Answer 1: `std::print`

Type safety, checked at compile time. Can format custom types (with `std::formatter`) Uses python formatting Very concise and easy to use.

Question 2: `auto`

What is the advantage of `auto` and type deduction in modern C++?

Answer 2: `auto`

Prevent types mismatches.
Easy use for templates
clean syntax

Question 3: Passing by _

What is the difference between passing by value, reference, and const reference?

Answer 3: `auto`

value complete copy reference use a pointer to edit the original variable const ref used like a read only of the original variable

Question 4: bindings

What are structured bindings and when would you use them?

Answer 4: `auto`

Structured binding unpacks and object (tuple, array, etc..) into named variables. You have direct access with `auto&` to update the original variable. Used for readability for complicated things like tuples or maps.

Question 5: Ranges

What are C++23 ranges and views?

Answer 5: auto

Question 6: constexpr

What does constexpr mean and when is it evaluated?

Answer 6: auto

Question 7: Vector vs Arrays

What is the difference between `std::vector` and raw arrays?

Answer 7: auto

Question 8: string_view

What is `std::string_view` and why is it more efficient than `std::string`?

Answer 8

Question 9: ranges

What is the purpose of `#include <<ranges>`

Question 10: lifetime

What is the lifetime of a temporary returned from a function?